1. Scope Matching—Cedar first considers policies where the principal, action, and resource match the request's principal, action, and resource. If no policy matches based on scope, the request is implicitly denied.
2. Condition Evaluation—Cedar then evaluates the conditions of any scoped-matching policies using attribute lookups and contextual values. Every `when` condition must evaluate to `true` and every `unless` condition must evaluate to `false`.

Only policies that pass both stages contribute their effect to the final decision.

DATA TYPES AND ACCESS

Cedar enforces a strongly typed, deterministic evaluation model, purpose-built for secure, predictable authorization logic.

Every value in Cedar must have a clearly defined type. These types are declared in the policy schema. This clarity prevents casting errors that could lead to subtle breaches. For example, comparing `"123"` (string) to `123` (number) would be a type mismatch, not a successful match, helping developers catch logic errors early. Cedar's type checker can also flag incorrect set literals (such as mixing types) before deployment, reducing surprises at runtime.

Cedar utilizes intuitive dot notation for accessing data fields:

```
principal.department
resource.owner
context.device.is_managed
```

This syntax mirrors how you reference object properties in everyday programming. Whether you're checking Bob's department or verifying the device's trust status, dot notation makes the code clear and easy to understand. Cedar also supports bracket syntax, which can be combined with dot notation. For example:

```
context["device"].is_managed
```